

Fine-tuning de BERT: un cerebro, tres tareas

Embeddings contextuales que resuelven la polisemia, y cómo adaptarlos a embeddings, clasificación y NER

Ramsés Alejandro Camas Nájera

Universidad Politécnica de Chiapas · 9º cuatrimestre · Semana 8, Sesión 4 · Junio 2026

El límite de la sesión pasada: la polisemia

Los embeddings estáticos (FastText) dan UN vector por palabra. “Banco” recibe el mismo vector aunque signifique cosas distintas.

Estático (Sesión 3)

`vec("banco") = [0.3, ...]`
siempre el mismo

Una palabra, un vector. El contexto de la oración se ignora.

Contextual (hoy: BERT)

`vec("banco" | "parque") ≠`
`vec("banco" | "dinero")`

El vector se calcula leyendo toda la oración. El significado depende del contexto.

La idea central de hoy: BERT lee la oración completa (en ambas direcciones) antes de asignar un vector a cada palabra. El mismo token recibe vectores distintos según su contexto. **Eso resuelve la polisemia — y abre tres aplicaciones.**

Agenda — un backbone, tres cabezas



Bloque 1 — BERT y embeddings

Qué es BERT, cómo se preentrenó, el paradigma de fine-tuning y Sentence-BERT para similitud.

~60 min



Bloque 2 — Clasificación

Adaptar BERT para clasificar texto. Clasificar las noticias chiapanecas por tema.

~55 min



Bloque 3 — NER

Reconocimiento de entidades: detectar lugares, organizaciones y personas en el corpus.

~50 min



Síntesis + cierre

El mismo modelo, tres cabezas distintas. El puente a RAG y la Unidad 3.

~15 min

01

BERT y embeddings

El modelo que lee en dos direcciones, y el paradigma que cambió el NLP: preentrenar una vez, adaptar mil veces.

- Qué es BERT: contexto bidireccional
- Cómo se preentrenó (masked LM)
- El paradigma de fine-tuning
- Sentence-BERT para similitud

Qué es BERT

BERT (2018) es un transformer que lee la oración completa, en ambas direcciones, y produce un vector contextual por token.



Bidireccional

Mira las palabras a la izquierda Y a la derecha a la vez. “Banco” se interpreta con todo su contexto, no solo lo anterior.



Atención

Cada palabra “pone atención” a las demás según su relevancia. Así pesa qué partes de la oración importan para cada token.



Vectores contextuales

La salida es un vector por token que depende de toda la frase. El mismo token cambia de vector según el contexto.

BERT = Bidirectional Encoder Representations from Transformers. Para el español hay versiones como BETO o modelos basados en RoBERTa entrenados por el BSC. No lo entrenamos desde cero — partimos de uno preentrenado.

Cómo aprendió: masked language modeling

BERT se preentrena con un juego de adivinanzas auto-supervisado: se ocultan palabras al azar y el modelo aprende a predecirlas leyendo el resto.



predice: “sequía”

Por qué bidireccional

Para adivinar la palabra oculta usa el contexto de AMBOS lados. Por eso BERT entiende cada palabra en su frase completa.

Datos infinitos y gratis

Cualquier texto sirve como ejercicio: la respuesta es la palabra que se ocultó. Sin anotación humana, como word2vec.

El paradigma: preentrenar una vez, adaptar mil



Preentrenamiento

- Una sola vez, carísimo (millones de oraciones, semanas de GPU).
- Lo hace Google / BSC, no ustedes.
- Resultado: un modelo que ya “sabe” español.



Fine-tuning

- Barato y rápido (cientos de ejemplos, minutos de GPU).
- Lo hacen ustedes, en su RTX 3090.
- Ajusta el modelo a SU tarea específica.

Por qué esto lo cambió todo

Antes, cada tarea de NLP se entrenaba desde cero con miles de ejemplos etiquetados. Con transfer learning, el modelo ya trae el conocimiento del idioma; solo le enseñan la tarea. **Las mismas representaciones sirven para embeddings, clasificación y NER — solo cambia la cabeza.**

BERT para embeddings: Sentence-BERT



El problema: BERT “crudo” no da buenos vectores de oración para comparar por coseno. Comparar dos frases requería pasarlas juntas por BERT — inviable para buscar en miles de documentos.

La solución: Sentence-BERT

- Se afina BERT con pares de oraciones “similares / distintas” usando una red siamesa.
- Aprende a poner cerca lo que significa lo mismo y lejos lo distinto (contrastive).
- Resultado: un vector por oración listo para coseno, como en el Lab 5 pero mucho mejor.

Para su dominio (español asistivo)

- Partan de un Sentence-BERT multilingüe o en español ya entrenado.
- Afínenlo con pares de su dominio (consulta ↔ documento relevante).
- El buscador semántico del Lab 5 mejora sin cambiar el coseno: solo mejoran los vectores.

Lab 6a — Afinar un encoder de oraciones



Fine-tuning de un Sentence-BERT para mejorar el buscador semántico del Lab 5 sobre su corpus.

1

Línea base — medir el buscador del Lab 5b (FastText promediado) con sus métricas del Lab 3. Ese es el número a superar.

2

Datos de entrenamiento — construir pares (consulta, documento relevante) a partir de sus grels del Lab 3.

3

Fine-tuning — afinar un Sentence-BERT en español con esos pares (sentence-transformers); pocas épocas, learning rate bajo.

4

Medir la mejora — re-evaluar con las mismas métricas. ¿Subió el nDCG frente a FastText? Documenten el delta.

Con 14 documentos el margen es chico: lo importante es el método, no el número. La defensa pregunta por qué pocas épocas y LR bajo.

02

Clasificación

La cabeza más simple: tomar el resumen de la oración y predecir una etiqueta. Aquí, el tema de la noticia.

- BERT + una capa lineal
- El token [CLS] como resumen
- Fine-tuning con poco dato
- Clasificar las noticias del corpus

BERT + una capa: clasificar una oración

BERT antepone un token especial [CLS]. Su vector de salida resume la oración entera; una capa lineal lo convierte en una etiqueta.



El token [CLS]

Es un token “resumen” que BERT aprende a llenar con la información de toda la oración. Su vector final representa la frase completa.

La cabeza de clasificación

Una sola capa lineal + softmax mapea el vector [CLS] a probabilidades sobre las clases. Es lo único “nuevo” que se entrena casi de cero.

Fine-tuning para clasificar — con datos reales

Callback al Lab 4: los grupos que K-Means descubrió (clima, agro, turismo, servicios) se vuelven las etiquetas. Pasamos de descubrimiento no supervisado a clasificación supervisada sobre el mismo corpus.

Datos y splits

Etiquetar las noticias por tema; partir en train / validación / test. Nunca medir en lo que se entrenó.

Hiperparámetros

LR bajo ($\sim 2e-5$), 2–4 épocas, warmup. Más épocas = overfitting con corpus chico.

Métricas

Accuracy y F1-macro (importa por clase, no solo el total). Vigilar el desbalance de temas.

El riesgo real con 14 documentos: el modelo memoriza en vez de aprender. Por eso el split honesto y pocas épocas importan más que la arquitectura.

Lab 6b — Clasificar noticias por tema



Fine-tuning de BERT en español para asignar a cada noticia su tema, usando los grupos del Lab 4 como etiquetas.

1

Etiquetar y dividir — asignar tema a cada documento (partiendo del Lab 4); split estratificado train/val/test.

2

Tokenizar — con el tokenizer del modelo (BETO o multilingüe); padding y truncation.

3

Fine-tuning — AutoModelForSequenceClassification + Trainer; LR 2e-5, 3 épocas, evaluar en validación.

4

Evaluar y analizar errores — accuracy y F1-macro en test; matriz de confusión: ¿qué temas se confunden y por qué?

La matriz de confusión es la defensa: deben explicar cada error, no solo reportar el accuracy.

03

NER

La cabeza más fina: en vez de una etiqueta por oración, una etiqueta por palabra. Detectar entidades en el texto.

- Qué es NER y el esquema BIO
- BERT por token + alineación de subpalabras
- Métricas a nivel de entidad
- Entidades en el corpus chiapaneco

Reconocimiento de entidades nombradas

NER detecta y clasifica los nombres propios del texto: lugares, organizaciones, personas, fechas. Es clasificación, pero por token.



El esquema BIO

B-XXX (Begin) inicio de una entidad de tipo XXX **I-XXX** (Inside) continuación de la entidad
O (Outside) no es entidad. Ejemplo: “San Cristóbal de las Casas” = B-LOC I-LOC I-LOC I-LOC I-LOC

BERT por token, y el problema de las subpalabras

Para NER se usa el vector de CADA token (no solo [CLS]); una capa lineal predice la etiqueta BIO de cada uno.



El gotcha: BERT parte las palabras en subpalabras. “Tuxtla” puede volverse [“tux”, “##tla”]. ¿A cuál subpalabra le asignan la etiqueta B-LOC? Hay que alinear etiquetas con subpalabras o el entrenamiento se rompe.

La convención

Etiquetar solo la PRIMERA subpalabra de cada palabra; las demás se marcan para ignorar en la pérdida (-100).

Por qué importa

Si se ignora, el modelo aprende etiquetas inconsistentes y las métricas mienten. Es el error #1 en NER con BERT.

En la práctica

La función `word_ids()` del tokenizer dice qué subpalabra pertenece a qué palabra. Con eso se alinea.

La alineación subpalabra ↔ etiqueta es la parte que más cuesta en el Lab 6c. Es donde se va el tiempo de depuración.

Medir bien: a nivel de entidad, no de token

La trampa del accuracy

La mayoría de los tokens son “O”. Un modelo que predice siempre “O” tendría ~90% de accuracy por token... y cero entidades detectadas. El accuracy por token engaña.

La forma correcta

Precisión, recall y F1 a nivel de ENTIDAD completa: la entidad cuenta solo si se acertó su tipo y sus límites exactos. La librería sequeval lo calcula.

Entidades en el corpus chiapaneco



LOC

Tuxtla, Chiapas, San Cristóbal,
Soconusco



ORG

UPCh, IMSS, DIF, Protección Civil



MISC

Kaná, Cañón del Sumidero

Lab 6c — NER sobre el corpus en español

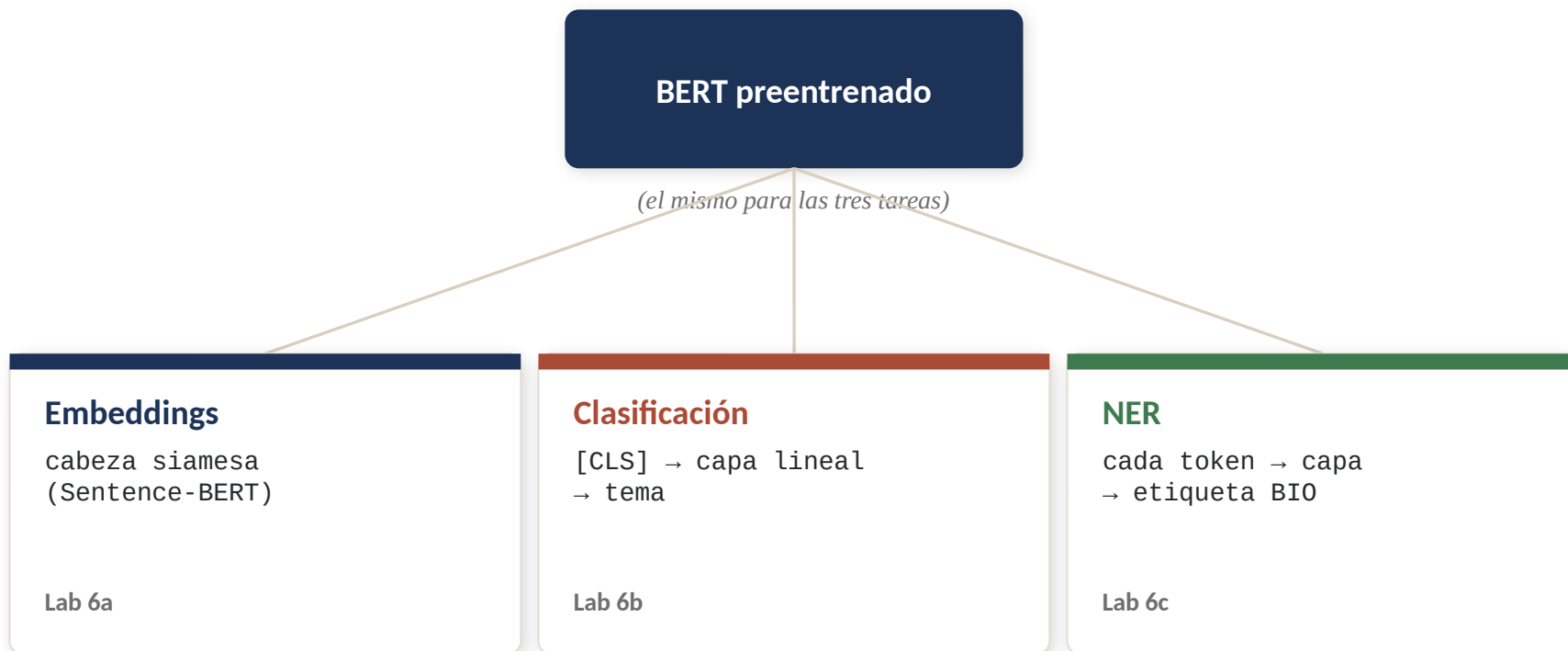


Fine-tuning de BERT para detectar lugares, organizaciones y entidades en las noticias chiapanecas.

- 1 **Etiquetar en BIO** — anotar entidades en un subconjunto del corpus (o partir de un dataset NER en español como CoNLL-2002).
- 2 **Alinear subpalabras** — tokenizar y mapear etiquetas a subpalabras con `word_ids()`; marcar con -100 las que se ignoran.
- 3 **Fine-tuning** — `AutoModelForTokenClassification` + `Trainer`; cabeza por token sobre el esquema BIO.
- 4 **Evaluar con segeval** — precisión/recall/F1 a nivel de entidad; inspeccionar qué entidades se escapan y por qué.

Para la defensa: muestren un caso donde la alineación de subpalabras, mal hecha, rompería las etiquetas.

Un mismo BERT, tres cabezas distintas



Lo que acaban de construir es la base de todo

Embeddings → recuperación

Sentence-BERT + base vectorial es la “R” de RAG: encontrar los documentos relevantes para una pregunta.

Clasificación → enrutamiento

Decidir intención, filtrar, moderar: clasificar es la pieza de control de cualquier sistema de NLP en producción.

NER → extracción

Estructurar texto libre en datos: entidades, fechas, montos. La base de pipelines de información.

La Unidad 3 ensambla estas piezas en sistemas reales

Transformers en profundidad · RAG (retrieval + generación) · privacidad y MLOps · el proyecto integrador. Todo se apoya en lo que hicieron estas cuatro sesiones.

Lo que sigue

Próxima unidad (Unidad 3)

- El transformer por dentro: atención, capas, posición.
- RAG: unir su buscador semántico con un modelo generativo.
- Privacidad, MLOps y el proyecto integrador final.

Para antes de la próxima clase

- Terminar Labs 6a, 6b y 6c y subirlos al repo (con AI_USAGE.md).
- Lectura: Devlin et al. 2018 (BERT) y Reimers & Gurevych 2019 (Sentence-BERT).
- Pensar: ¿qué pieza de hoy usarían en su proyecto integrador?

Fin de la Unidad 2. Del conteo de palabras al significado contextual: el viaje completo del texto al sentido.

Bibliografía de la sesión

-  **Devlin, J. et al. (2019).** BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL. arXiv:1810.04805.
-  **Reimers, N. & Gurevych, I. (2019).** Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP.
-  **Cañete, J. et al. (2020).** Spanish Pre-Trained BERT Model and Evaluation Data (BETO). PML4DC, ICLR.
-  **Tjong Kim Sang, E. (2002).** Introduction to the CoNLL-2002 Shared Task: Language-Independent NER (datasets en español).
-  **Jurafsky, D. & Martin, J. H. (2026).** Speech and Language Processing (3ª ed., borrador). — Caps. 11 (transformers/BERT) y 8 (NER y secuencias).